Module 9 Lab Session 3: Defensive RxJS and Real-Time Sync

Module	M9 Angular Advanced Concepts
Session	3 of 3
Exercises	Exercise 4: `exhaustMap`, `takeUntilDestroyed`, Exercise 5: SignalR live sync

Story Thread

It is the last day of midterm grading. Dawit has 120 final grades to submit before the 5 PM deadline. He fills in the grade form for a student and clicks **Submit Final Grades**. The button gives no visual feedback no spinner, no disabled state. After waiting two seconds with nothing happening on his slow connection, he clicks again. And again. And again.

He opens the browser console and discovers the Network tab shows **five** identical POST requests to `/api/grades`. The database now contains five duplicate grade records for the same student.

The root cause: each button click called `this.api.postGrade(payload).subscribe()` independently. Every call launched a separate HTTP request in parallel. The server had no way to know the client was rage-clicking; it processed all five faithfully.

This is not a server bug. This is a **client architecture failure**. And the fix lives in how we manage the RxJS event stream.

Exercise 4: The Rage-Click Defender (`exhaustMap`)

The Three Flattening Operators - When Each One Matters

Before writing code, understand the three RxJS operators that handle "a new event arrives while a previous HTTP request is still in flight":

Operator	What it does with the old request	Best use case
`switchMap`	**Cancels** the old request, starts the new one	**Search boxes:** cancel the slow "Smi" search when the user types "Smith"
`exhaustMap`	**Ignores** the new emission, finishes the old one	**Submit buttons:** drop rage-clicks while the first request is pending
`concatMap`	**Queues** the new one after the old one finishes	**Sequential data syncs:** process actions in strict order

For Dawit's grade submission, the correct choice is `exhaustMap`. While the first POST is in flight, any subsequent clicks are silently dropped. The first request completes, the grade is saved once, and Dawit moves to the next student.

Using `switchMap` here would be dangerous: it cancels the in-flight request on the client, but the server may have already processed and committed the grade before the cancellation frame arrives leading to a saved grade with no client confirmation.

Step 1: Generate the Component and Service

Generate the grade submission feature component and its supporting HTTP service:

```
ng generate service services/grade --type=service
ng generate component features/grade-submission --type=component
```

Step 2: Implement the Grade Service

Open `src/app/services/grade.service.ts` and implement the HTTP client interface using Angular 22's `@Service()` decorator:

```
export interface GradePayload {
  studentId: number;
  courseId: number;
  score: number;
}
```

```
@Service()
export class GradeService {
  private http = inject(HttpClient);

  postGrade(payload: GradePayload): Observable<{ id: string; success: b
oolean }> {
    return this.http.post<{ id: string; success: boolean }>('/api/grade
s', payload);
  }
}
```

Step 3: Implement the Guarded Component Class (Reactive Form)

Open `src/app/features/grade-submission/grade-submission.component.ts`. Import `ReactiveFormsModule`, `FormBuilder`, and `Validators` alongside Angular Material components. Construct an explicit `gradeForm` group and set up the Subject-based event stream protected by `exhaustMap`:

```
@Component({
  selector: 'tms-grade-submission',
  standalone: true,
  imports: [
    // Do import the necessary modules and components
  ],
  templateUrl: './grade-submission.component.html'
})
export class GradeSubmissionComponent {
  private api = inject(GradeService);
  private fb = inject(FormBuilder);

  // Reactive Form definition with initial model values and validators
  gradeForm = this.fb.group({
    studentId: [101, [Validators.required, Validators.min(1)]],
    courseId: [302, [Validators.required, Validators.min(1)]],
    score: [88, [Validators.required, Validators.min(0),
Validators.max(100)]]
  });

  isSubmitting = false;
  submissionStatus = '';

  // A Subject is a manual event stream — template clicks push payloads
into it
  private submitClick$ = new Subject<GradePayload>();
  constructor() {
    this.submitClick$
```

```
      .pipe(
        // exhaustMap: while the inner HTTP observable is active,
        // ALL new emissions from submitClick$ are silently dropped.
        // Dawit can click 50 times — only ONE POST request fires.
        exhaustMap(payload => {
          this.isSubmitting = true;
          this.submissionStatus = 'Submitting grade to server...';
          return this.api.postGrade(payload);
        }),

        // takeUntilDestroyed: automatically unsubscribes when Angular
        // destroys this component, preventing memory leaks.
        // Placed inside constructor to inherit the active injection
context.
        takeUntilDestroyed()
      )
      .subscribe({
        next: result => {
          this.isSubmitting = false;
          this.submissionStatus = `Grade saved successfully! Record ID:
${result.id}`;
        },
        error: err => {
          this.isSubmitting = false;
          this.submissionStatus = `Submission failed: ${err.message ||
'Server error'}`;
        }
      });
  }

  // The template form submit handler pushes valid values into the
protected stream
  onSubmit() {
    if (this.gradeForm.valid) {
      const rawValue = this.gradeForm.getRawValue();
      this.submitClick$.next({
        studentId: Number(rawValue.studentId),
        courseId: Number(rawValue.courseId),
        score: Number(rawValue.score)
      });
    }
  }
}
```

Step 4: Build the Grade Submission Template (Reactive Form + Material + Tailwind)

Open `src/app/features/grade-submission/grade-submission.component.html` and bind the `[formGroup]="gradeForm"` with `formControlName` bindings, validation error messages (`<mat-error>`), and button disability states:

```
<div class="max-w-md mx-auto my-8">
  <mat-card class="shadow-xl rounded-2xl bg-slate-900 border border-sla
te-800 text-slate-100 p-6">
    <mat-card-header class="mb-4">
      <mat-card-title class="text-xl font-bold text-slate-100">Grade Su
bmission Form</mat-card-title>
      <mat-card-subtitle class="text-slate-400 text-sm">Instructor Midt
erm Grading</mat-card-subtitle>
    </mat-card-header>

    <form [formGroup]="gradeForm" (ngSubmit)="onSubmit()">
      <mat-card-content class="space-y-4">
        <mat-form-field appearance="outline" class="w-full">
          <mat-label>Student ID</mat-label>
          <input matInput type="number" formControlName="studentId" />
          @if (gradeForm.controls.studentId.hasError('required')) {
            <mat-error>Student ID is required</mat-error>
          }
        </mat-form-field>

        <mat-form-field appearance="outline" class="w-full">
          <mat-label>Course ID</mat-label>
          <input matInput type="number" formControlName="courseId" />
          @if (gradeForm.controls.courseId.hasError('required')) {
            <mat-error>Course ID is required</mat-error>
          }
        </mat-form-field>

        <mat-form-field appearance="outline" class="w-full">
          <mat-label>Score (0-100)</mat-label>
          <input matInput type="number" formControlName="score" />
          @if (gradeForm.controls.score.hasError('min') || gradeForm.co
ntrols.score.hasError('max')) {
            <mat-error>Score must be between 0 and 100</mat-error>
          }
        </mat-form-field>

        @if (isSubmitting) {
          <div class="flex justify-center py-3">
            <mat-spinner diameter="32"></mat-spinner>
          </div>
```

```
        }

        @if (submissionStatus) {
          <div class="mt-4 p-3 rounded-lg bg-slate-800 text-sky-400 tex
t-sm font-medium border border-slate-700">
            {{ submissionStatus }}
          </div>
        }
      </mat-card-content>

      <mat-card-actions class="mt-4">
        <button
          mat-raised-button
          color="primary"
          type="submit"
          [disabled]="gradeForm.invalid || isSubmitting"
          class="w-full py-3 text-base font-semibold">
          Submit Final Grade
        </button>
      </mat-card-actions>
    </form>
  </mat-card>
</div>
```

Step 5: Add Route Registration

Open `src/app/app.routes.ts` and add the lazy-loaded route:

```
{
  path: 'grade-submission',
  loadComponent: () =>
    import('./features/grade-submission/grade-submission.component')
      .then(m => m.GradeSubmissionComponent)
}
```

Verify Exercise 4

1. Start the dev server (`ng serve`) and navigate to `http://localhost:4200/grade-submission`.
2. Try submitting invalid values (e.g. score set to 150 or empty student ID) — observe reactive `<mat-error>` messages and the disabled Submit button.
3. Set the Chrome DevTools network throttling dropdown to **Slow 3G** (simulating slow server response times).
4. Click the **Submit Final Grade** button rapidly **10 times in a row**.
5. Inspect the Network tab. You will observe exactly **one** POST request to `/api/grades`, while the Material spinner provides visual loading feedback.

The subsequent 9 clicks were silently ignored by `exhaustMap` while the first request was in flight.

Exercise 5: Real-Time Sync with SignalR

The Problem with Polling

Right now, if another instructor approves an enrollment on a different computer, Dawit's dashboard does not update until he manually refreshes the page. One approach would be to poll the API every few seconds with `setInterval`. But polling wastes bandwidth, delays updates, and does not scale when you have hundreds of connected clients.

SignalR solves this by opening a persistent WebSocket connection between the browser and the server. When a server-side event occurs (enrollment approved, grade submitted, course status changed), the server pushes the event directly to every connected client in real time.

In M7 Session 3, you built the backend `TmsHub` with a strongly-typed client interface (`ITmsHubClient`) that already sends `ReceiveTranscriptReady` and `ReceiveGradePosted`. Now you will extend that same hub contract with enrollment status broadcasts, wire the enrollment approval endpoint to push events, and build the Angular client that subscribes to them.

Step 1: Extend the Backend Hub Client Interface

In M7 Session 3, you created `TmsApi.Application/Hubs/ITmsHubClient.cs` with three events (`ReceiveTranscriptReady`, `ReceiveCourseUpdate`, `ReceiveGradePosted`). Open that file and add the enrollment status event:

```
// File: TmsApi.Application/Hubs/ITmsHubClient.cs
public interface ITmsHubClient
{
// New: broadcast enrollment status changes to all connected clients
Task ReceiveEnrollmentStatusUpdated(string enrollmentId, string status);
}
```

Because `TmsHub` extends `Hub<ITmsHubClient>`, this new method is immediately available on `Clients.All`, `Clients.Group(...)`, and every other SignalR target with full compile-time type safety. No magic strings.

Step 2: Broadcast from the Enrollment Approval Endpoint

Open your enrollment controller (for example, `TmsApi.Api/Controllers/V2/EnrollmentsController.cs`). Inject

`IHubContext<TmsHub, ITmsHubClient>` and broadcast the status change after the database commit succeeds:

```
// File: TmsApi.Api/Controllers/V2/EnrollmentsController.cs

public class EnrollmentsController(
    /* your existing dependencies */
    IHubContext<TmsHub, ITmsHubClient> hubContext) : ControllerBase
{
    [HttpPost("{id}/approve")]
    public async Task<IActionResult> Approve(string id, CancellationTok
en ct)
    {
        // Your existing approval logic ...

        // After the database commit succeeds, broadcast to all connect
ed Angular clients
        await hubContext.Clients.All
            .ReceiveEnrollmentStatusUpdated(id, "Approved");

        return NoContent();
    }
```

Notice the call uses `hubContext.Clients.All` not `Clients.Group(...)`. For enrollment status changes visible to all instructors on the dashboard, broadcasting to all connected clients is the correct choice.

Step 3: Install SignalR Client Package (Angular)

Install the official Microsoft SignalR client library in your Angular project:

```
npm install @microsoft/signalr
```

Step 4: Configure the Dev Server Proxy

The Angular dev server runs on `localhost:4200`. Your .NET API runs on `localhost:5000` (or `5001` for HTTPS). Without a proxy, relative URLs like `/hubs/tms` and `/api/enrollments` will hit `localhost:4200` which returns 404 because there is no backend there.

Create `proxy.conf.json` in your Angular project root (next to `angular.json`):

```
{
  "/api": {
    "target": "http://localhost:5000",
    "secure": false,
    "changeOrigin": true
  },
  "/hubs": {
```

```
    "target": "http://localhost:5000",
    "secure": false,
    "ws": true
  }
}
```

The `"ws": true` flag on the `/hubs` entry is critical, it tells the proxy to upgrade the connection to WebSocket protocol. Without it, SignalR's WebSocket handshake fails silently and falls back to long polling (much slower, higher server load).

Wire the proxy into `angular.json` under the `serve` options:

```
"serve": {
  "options": {
    "proxyConfig": "proxy.conf.json"
  }
}
```

Restart `ng serve` after this change (proxy config is only read at startup).

The proxy makes your Angular dev server forward all `/api` and `/hubs` traffic to the .NET backend so both servers appear as the same origin to the browser.

Step 5: Build the Live Sync Service

Generate the service (`ng generate service services/live-sync`). Open `src/app/services/live-sync.service.ts` and implement the production SignalR connection manager .

```
export interface EnrollmentStatusEvent {
  id: string;
  status: 'Pending' | 'Approved' | 'Rejected';
}

@Service()
export class LiveSyncService {
  private platformId = inject(PLATFORM_ID);
  private connection: HubConnection | null = null;
  private eventsSubject = new Subject<EnrollmentStatusEvent>();

  // Expose events as an observable — the store will subscribe to this
  events$ = this.eventsSubject.asObservable();

  // Connection state signal for UI status feedback
  connectionState = signal<'connected' | 'reconnecting' |
'disconnected'>('disconnected');

  connect() {
```

```
    // Guard against duplicate connections if called more than once
    if (this.connection) return;

    // SignalR uses WebSocket which only exists in browsers, not on the
Node.js server.
    // If SSR is enabled (Extension 1), this method runs during server
render — skip it.
    if (!isPlatformBrowser(this.platformId)) return;

    // Same hub URL and reconnect strategy you tested in M7 Session 3
browser DevTools
    this.connection = new HubConnectionBuilder()
      .withUrl('/hubs/tms')
      .withAutomaticReconnect([0, 2000, 10000, 30000])
      .build();

    // The event name matches the ITmsHubClient method you just added
on the backend.
    // SignalR strongly-typed hubs send the method name as the event
name automatically.
    this.connection.on(
      'ReceiveEnrollmentStatusUpdated',
      (enrollmentId: string, status: 'Pending' | 'Approved' |
'Rejected') => {
        this.eventsSubject.next({ id: enrollmentId, status });
      }
    );

    this.connection.onreconnecting(() =>
this.connectionState.set('reconnecting'));
    this.connection.onreconnected(() =>
this.connectionState.set('connected'));
    this.connection.onclose(() =>
this.connectionState.set('disconnected'));

    this.connection
      .start()
      .then(() => this.connectionState.set('connected'))
      .catch(err => console.error('SignalR connection error:', err));
  }
}
```

The event name `'ReceiveEnrollmentStatusUpdated'` maps exactly to the `ITmsHubClient.ReceiveEnrollmentStatusUpdated` method you added in Step 1. When the backend calls `hubContext.Clients.All.ReceiveEnrollmentStatusUpdated(id, "Approved")`, SignalR serialises the method name as the event identifier. The Angular client

listens for that same string. This is why strongly-typed hubs matter, the C# compiler catches typos before they become silent runtime mismatches.

Step 6: Bridge Live Events into the SignalStore

`LiveSyncService` exposes events as an observable (`events$`) rather than directly mutating state. This is a deliberate architectural choice: the **hub service** manages connection transport, while the **store** manages state mutations.

Open `src/app/store/enrollment.store.ts`. Update the imports and add `listenForLiveUpdates` inside `withMethods`:

```
export const EnrollmentStore = signalStore(
  { providedIn: 'root' },
  withEntities<Enrollment>(),
  withMethods((
    store,
    api = inject(EnrollmentService),
    sync = inject(LiveSyncService)
  ) => ({

    // Listens to SignalR live sync stream and updates store state
automatically
    listenForLiveUpdates: rxMethod<void>(
      pipe(
        tap(() => sync.connect()),
        switchMap(() => sync.events$),
        tap(event => {
          patchState(
            store,
            updateEntity({ id: event.id, changes: { status:
event.status } })
          );
        })
      )
    ),

    // ... your existing loadEnrollments() and approveEnrollment()
methods
  }))
);
```

Step 7: Activate Live Sync on App Startup

Open `src/app/app.component.ts` (or `instructor-dashboard.component.ts`) and trigger the live listener:

```
export class AppComponent implements OnInit {
  private store = inject(EnrollmentStore);
```

```
  ngOnInit() {
    this.store.loadEnrollments();
    this.store.listenForLiveUpdates();
  }
}
```

Verify Exercise 5 (Real-Time Live Sync)

1. Start both the .NET backend (`dotnet run --project TmsApi.Api`) and the Angular dev server (`ng serve`).
2. Open two separate browser tabs side by side (`http://localhost:4200/enrollments` in Tab 1, and `http://localhost:4200/dashboard` in Tab 2).
3. In Tab 1, approve a pending enrollment in the table grid.
4. Watch Tab 2: the pending count on the dashboard updates instantly without a page refresh or manual API poll. The .NET `EnrollmentsController.Approve` called `hubContext.Clients.All.ReceiveEnrollmentStatusUpdated(id, "Approved")`, which pushed the event through the WebSocket to every connected Angular `LiveSyncService` → `EnrollmentStore` → component binding.
5. (Optional) Open Chrome DevTools → Console and verify the connection state: you should see no errors and the `connectionState` signal reads `'connected'`. Stop the .NET backend briefly — the console logs "Reconnecting…" then "Reconnected" when you restart it, proving `withAutomaticReconnect` works.

What You Just Built

Open `http://localhost:4200/grade-submission` on Slow 3G throttling. Rapidly click **Submit Final Grade** 10 times. Exactly **one** HTTP request fires to `/api/grades`, accompanied by a Reactive Form, Tailwind styling, and a Material progress spinner. Open two browser windows side by side. Approve an enrollment in Tab 1 — Tab 2 updates instantly via SignalR push without polling.

Here is what you can show a teammate or facilitator to demonstrate competency:

- **Reactive Form + Material UI proof:** Form utilizes `ReactiveFormsModule`, `FormBuilder`, `Validators`, `<mat-error>` messages, and disabled button state.

- **RxJS `exhaustMap` proof:** On Slow 3G network throttling, rapid button double-clicks result in exactly one network request in the DevTools Network tab.
- **RxJS Memory Safety proof:** Code uses `takeUntilDestroyed()` in constructor streams, preventing dangling subscription leaks.
- **SignalR Full-Stack Proof:** `LiveSyncService` connects to the M7 `TmsHub` (`/hubs/tms`) and listens for `ReceiveEnrollmentStatusUpdated;` the strongly-typed `ITmsHubClient` method you extended on the backend. Status changes broadcast from the .NET `EnrollmentsController` update all open Angular clients instantly.
- **Architecture proof:** Separate concerns between transport (`LiveSyncService`) and state management (`EnrollmentStore`).

Module 9 Complete

You have built a high-performance, real-time Angular 22 single-page application:

- **Session 1:** Centralized state management with `NgRx SignalStore` eliminating data drift between components.
- **Session 2:** Performance optimization with `@defer` code splitting and enterprise Material grids (`MatTable`, `MatSort`, `MatPaginator`).
- **Session 3:** Defensive RxJS with `exhaustMap` to prevent duplicate submissions, and real-time push events via SignalR (`LiveSyncService`).

Next module: In **M10 Full-Stack Integration**, you will connect this Angular client to the .NET API under real browser security rules configuring CORS policies, HttpOnly authentication cookies, XSRF protection, functional HTTP interceptors, route guards, and optimistic UI rollback. Everything built here is your frontend foundation.